

Building a Jupyter Kernel

How Python can bring another language to the notebook

Using case study of IForth - a kernel for Forth language

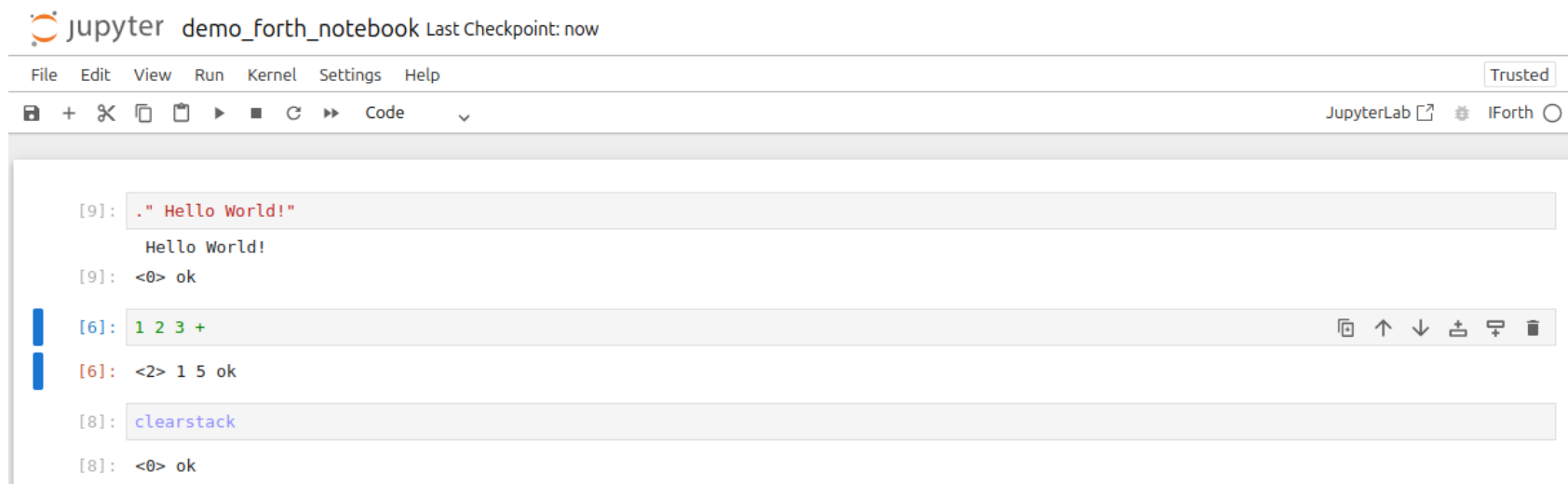
Jupyter Notebook Examples - Python and Forth



The image shows a Jupyter Notebook interface for a Python kernel. The title bar reads "jupyter demo_forth_notebook Last Checkpoint: 2 minutes ago". The menu bar includes File, Edit, View, Run, Kernel, Settings, and Help. The toolbar shows icons for saving, adding, deleting, and running code. The code area contains two cells. The first cell has the code `print("Hello World!")` and the output "Hello World!". The second cell has the code `2 + 3` and the output `5`. The status bar at the bottom right indicates "JupyterLab Python 3 (ipykernel)".

```
[1]: print("Hello World!")
Hello World!

[2]: 2 + 3
[2]: 5
```



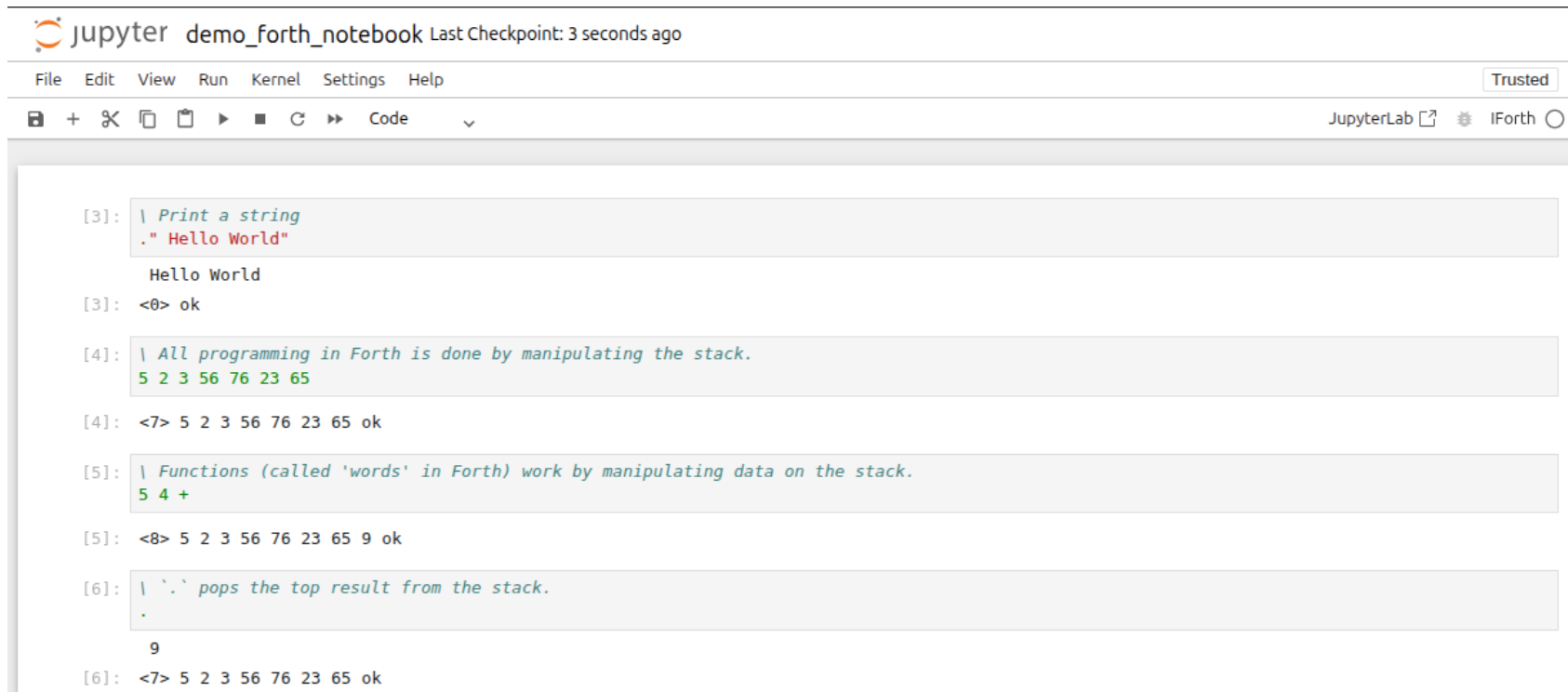
The image shows a Jupyter Notebook interface for an IForth kernel. The title bar reads "jupyter demo_forth_notebook Last Checkpoint: now". The menu bar includes File, Edit, View, Run, Kernel, Settings, and Help. The toolbar shows icons for saving, adding, deleting, and running code. The code area contains three cells. The first cell has the code `." Hello World!"` and the output "Hello World!". The second cell has the code `1 2 3 +` and the output `<0> ok`. The third cell has the code `clearstack` and the output `<0> ok`. The status bar at the bottom right indicates "JupyterLab IForth".

```
[9]: ." Hello World!"
Hello World!
[9]: <0> ok

[6]: 1 2 3 +
[6]: <2> 1 5 ok

[8]: clearstack
[8]: <0> ok
```

Forth in 30 seconds



The screenshot shows a JupyterLab interface with a notebook titled "demo_forth_notebook". The top bar includes the Jupyter logo, the notebook name, and a "Last Checkpoint: 3 seconds ago" status. Below the top bar is a menu bar with "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". To the right of the menu bar is a "Trusted" button. Below the menu bar is a toolbar with icons for file operations, a "Code" button, and a "JupyterLab" button. The notebook content consists of six code cells, each starting with a prompt like "[3]:". Each cell contains a Forth code snippet followed by its output. The code snippets are: 1. A comment "Print a string" followed by a string literal "Hello World". 2. A comment "All programming in Forth is done by manipulating the stack." followed by a stack of numbers. 3. A comment "Functions (called 'words' in Forth) work by manipulating data on the stack." followed by a stack of numbers and an addition operation. 4. A comment "pops the top result from the stack." followed by a stack of numbers and a pop operation. The outputs show the stack state after each operation.

```
[3]: | Print a string
    ." Hello World"
    Hello World
[3]: <0> ok

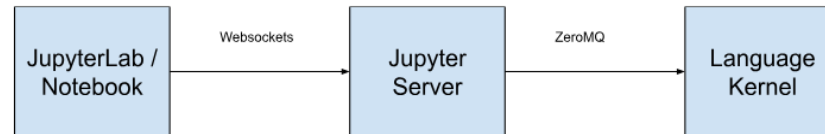
[4]: | All programming in Forth is done by manipulating the stack.
    5 2 3 56 76 23 65
[4]: <7> 5 2 3 56 76 23 65 ok

[5]: | Functions (called 'words' in Forth) work by manipulating data on the stack.
    5 4 +
[5]: <8> 5 2 3 56 76 23 65 9 ok

[6]: | '.' pops the top result from the stack.
    .
    9
[6]: <7> 5 2 3 56 76 23 65 ok
```

See more syntax examples: <https://learnxinyminutes.com/forth>

The message flow



The Jupyter Server communicates with the kernel over 5 ZeroMQ channels:

- Shell: Main request/reply loop, frontend sends code to run here.
- IOPub: Broadcast channel. Kernel can publish stdout, stderr, HTML, etc.
- Stdin: Kernel can request user input.
- Control: High-priority commands (eg. Shutdown, Interrupt) that bypass execution queue.
- Heartbeat: Simple ping/pong socket, check kernel is still alive.

Source: <https://www.datahaskell.org/blog/2025/11/25/a-tale-of-two-kernels.html#the-jupyter-kernel-architecture>

Subclass Jupyter Kernel

```
class IForth(Kernel):
    """Jupyter kernel for Forth language."""
    implementation = "forth"
    implementation_version = __version__

    gforth = Gforth()  # spawns subprocess: long-lived gforth interpreter

    def answer_text(self, text: str, stream: Literal["stdout", "stderr"]) -> None:
        """Send text response to Jupyter cell."""
        logger.info("Answering to Jupyter: %s", text)
        self.send_response(
            self.iopub_socket, "stream", {"name": stream, "text": text}
        )

    def answer_expression_value(self, expression_value: str) -> None:
        self.send_response(self.iopub_socket, "execute_result", {
            "execution_count": self.execution_count,
            "data": {
                "text/plain": expression_value,
            },
            "metadata": {}
        })

    @override
    async def do_execute(  # do_execute() can also be synchronous
        self,
        code: str,
        silent: bool,
        store_history: bool = True,
        user_expressions = None,
        allow_stdin = None,
        *,
        cell_meta = None,
        cell_id = None,
    ) -> dict[str, Any]:
        """Execute Forth code."""
        logger.info("do_execute (silent=%s): %s", silent, code)
        stack_output = await self.gforth.exec(code, self.answer_text)
        if stack_output is not None:
            self.answer_expression_value(stack_output)
        return {
            'status': 'ok',
            'execution_count': self.execution_count,  # Jupyter's tracked cell count
            'payload': [],
            'user_expressions': {}
        }
```

Register the custom kernel with Jupyter

```
# dynamically generate kernel.json so that correct Python path is used
kernel_spec = {
    "argv": [
        # path to python (of the venv where forth kernel is installed)
        # allows it to run even if Jupyter is installed in a different venv
        sys.executable,
        "-m",
        "forth_kernel",
        "-f",
        "{connection_file}"
    ],
    "display_name": "IForth",
    "language": "Forth",
    "codemirror_mode": "text",
    "interrupt_mode": "message",
    "name": "Forth"
}

script_dir = Path(__file__).parent.resolve()
logger.info('Script Dir: %s', script_dir)
with (script_dir / 'kernel.json').open('w') as f:
    json.dump(kernel_spec, f, indent=4)

try:
    install_kernel_spec(str(script_dir), 'forth', replace=True, user=args.user)
    logger.info('Successfully installed jupyter kernel for Forth.')
except PermissionError as e:
    logger.error(
        'Failed to install jupyter kernel for Forth: %s\n' +
        'Try installing as user instead of root: python -m forth_kernel.self_install --user',
        e
    )
except Exception as e:
    logger.error('Failed to install jupyter kernel for Forth: %s', e)
```

Automated Tests using module jupyter_kernel_test

```
class MyKernelTests(jupyter_kernel_test.KernelTests):
    kernel_name = "forth"
    language_name = "forth"
    file_extension = ".4th"

    code_hello_world = '." hello, world"'
    code_stderr = 'clearstack .' # stack underflow error
    code_execute_result = [
        {'code': '1 2 3 .', 'result': '<2> 1 2 ok'},
    ]

    def setUp(self):
        self.flush_channels()

    def test_multiline_cell_is_prompt(self):
        """Regression: each line used to cost a full read timeout."""
        start = time.monotonic()
        reply, _ = self.execute_helper(code='1 .\n2 .\n3 .\n4 .\n5 .')
        self.assertEqual(reply['content']['status'], 'ok')
        self.assertLess(time.monotonic() - start, 5)

    def test_long_output_not_truncated(self):
        """Regression: output was cut off when it exceeded the read timeout."""
        _, msgs = self.execute_helper(code=': countup 200 0 do i . loop ; countup')
        out = ''.join(m['content']['text'] for m in msgs)
        if m['msg_type'] == 'stream'
            and m['content']['name'] == 'stdout')
        self.assertIn('199', out)

if __name__ == "__main__":
    import unittest
    unittest.main()
```

Check out IForth for an example Jupyter kernel implementation.

And maybe try Forth - it's an interesting language :)

Repo: <https://github.com/sohang3112/iforth>

Thank You!

PS: This presentation was made using <https://docs.racket-lang.org/slideshow/>

Questions?